cairedine.finance

Guide d'implémentation

Module Intelligence IA

Synthèse Hybride : Métriques Quantitatives & SEC EDGAR

Spring AI · Gemini 2.0 Flash · Spring Modulith · PostgreSQL

12 prompts séquentiels — Ingestion Batch & Restitution JSON pour Angular

Architecture v3 (Enrichissement Quanti + Quali & Rapport PDF Frontend) — Mars 2026

1. Introduction & Objectifs

Ce document contient les **12 prompts ordonnés** nécessaires pour implémenter de bout en bout le module **Intelligence IA** dans le backend cairedine.finance. Chaque prompt est modulaire, autonome et produit un livrable compilable.

Objectif du module

Le module intelligence réalise une **fusion hybride** entre les métriques financières quantitatives calculées (module financialanalysis) et la couche qualitative issue des rapports réglementaires SEC EDGAR (10-K annuels et 10-Q trimestriels) :

- **Téléchargement automatique** des rapports 10-K (×2) et 10-Q (×4) depuis l'API officielle SEC EDGAR (gratuit, officiel).
- **Nettoyage & Extraction ciblée** des sections clés : *Risk Factors*, *MD&A;*, *Guidance* et *Supply Chain*.
- **Récupération des métriques quantitatives** : Marges d'exploitation, Net Debt / EBITDA, ROIC, FCF margin, modèles de valorisation.
- **Appel unique à Gemini 2.0 Flash** : Synthèse financière croisée mettant en perspective les chiffres réels et le discours du management.
- **Stockage PostgreSQL structuré** avec un TTL de 90 jours (réutilisation de l'infrastructure relationnelle existante).
- **Exposition REST optimisée** (GET /api/v1/insights/{ticker}) : Restitution d'un JSON consolidé ultra-rapide (< 50ms), permettant au **frontend Angular** de générer le rapport PDF côté client.

Ce que le module ne fait PAS

- **Pas de Chat en direct ni de Q&A;** : L'utilisateur n'interagit pas en temps réel avec le LLM.
- **Pas de VectorStore ni de RAG lourd** : Suppression de pgvector et des calculs d'embeddings.
- **Pas de MongoDB** : Base relationnelle PostgreSQL unique et centralisée.
- **Pas de génération PDF côté serveur** : Le backend Spring Boot reste une API REST performante et légère ; le rendu graphique et le téléchargement PDF sont confiés au client Angular.

2. Architecture Cible & Stack Technique

Phase 1 : Ingestion Asynchrone (Spring Modulith)	Phase 2 : Consultation & Export PDF (Angular)
• Événement TickerAddedEvent émis par la Watchlist. • IntelligenceEventListener déclenche le traitement asynchrone. • Fetch parallèle 10-K & 10-Q sur SEC EDGAR via SecEdgarAdapter. • Récupération des FullMetrics via IFinancialAnalysisService. • Nettoyage Tika & Extraction Regex des sections stratégiques. • Prompt de synthèse croisée envoyé à **Gemini 2.0 Flash**. • Sauvegarde du résumé JSON structuré en PostgreSQL (TTL 90j).	• GET /api/v1/insights/{ticker} : - Si ingestion en cours : 202 Accepted (Polling). - Si prêt : 200 OK avec InsightResponse JSON. • GET /api/v1/insights/{ticker}/status : statut d'indexation. • **Frontend Angular** : Consomme le JSON unique, affiche les tableaux/graphiques et compile le rapport PDF côté client (ex: via pdfmake ou jsPDF).

Composants & Technologies

Composant	Rôle	Technologie
SEC EDGAR	Source officielle des 10-K et 10-Q	API REST SEC (data.sec.gov / www.sec.gov)
CikResolverService	Résolution Ticker -> CIK 10 chiffres	Cache mémoire @PostConstruct
HtmlTextCleaner	Extraction texte brut depuis HTML	Spring AI Tika Document Reader
SectionExtractor	Isolation MD&A;, Risk Factors, Guidance	Regex sur titres officiels SEC
financialanalysis	Métriques quantitatives (marges, ratios)	Module interne (IFinancialAnalysisService)
Gemini 2.0 Flash	Synthèse financière croisée quanti/quali	Google AI Studio — Spring AI ChatClient
PostgreSQL	Stockage résumés consolidés (TTL 90j)	Spring Data JPA + JSON AttributeConverter
Angular Frontend	Rendu UI & Génération PDF client	Angular 19 / pdfmake / jsPDF

3. Récapitulatif des 12 Prompts d'Implémentation

Les prompts doivent être exécutés dans l'ordre séquentiel pour garantir la validation Modulith :

N°	Composant	Description
P01	pom.xml (Mise à jour)	Nettoyer dépendances : supprimer pgvector/MongoDB, garder Google GenAI et Tika.
P02	application.yaml (Mise à jour)	Configuration SEC EDGAR (base-url, user-agent) et summarization.
P03	Structure module intelligence	package-info.java Modulith, arborescence et squelettes de classes.
P04	Modèles domaine + Entité JPA	Records CompanyInsight (hybride quanti/quali), SecFiling et entité PostgreSQL.
P05	Interfaces de port (Domaine)	IDocumentSourcePort, ICompanyInsightRepository, IAiSummaryService.
P06	CikResolverService + RestClients	Cache CIK mémoire et configuration des deux RestClient SEC EDGAR.
P07	SecEdgarAdapter	Téléchargement des 10-K et 10-Q depuis EDGAR et parsing d'index.
P08	HtmlTextCleaner + SectionExtractor	Extraction Tika du texte brut et découpage regex des sections SEC.
P09	DocumentIngestionService + AiSummary	Pipeline unifié : injection IFinancialAnalysisService + prompt croisé Gemini + JPA.
P10	InsightController + DTOs Restitution	Endpoints GET /insights/{ticker} (200/202) servant le JSON consolidé pour Angular.
P11	Intégration événementielle Modulith	TickerAddedEvent -> IntelligenceEventListener -> Ingestion automatique.
P12	Tests unitaires & intégration	Tests Mockito (vérification prompt quanti/quali, pipeline, idempotence, controller).

PROMPT 01 (MISE À JOUR) — Dépendances Maven — pom.xml

■ Objectif :

Mettre à jour le pom.xml pour ne conserver que les dépendances Spring AI strictement nécessaires : Google GenAI (Gemini) et Tika. Supprimer pgvector et MongoDB qui ne sont plus requis.

■ Contexte :

Le module intelligence n'utilise pas de base vectorielle ni de base MongoDB. L'architecture repose sur l'API SEC EDGAR, l'injection des métriques calculées, l'appel unifié Gemini et le stockage relationnel PostgreSQL (JPA).

■ Instructions & Tâches :

Tu es un expert Spring Boot et Maven. Voici le pom.xml actuel du projet : [COLLER LE pom.xml]

TÂCHE 1 — Supprimer ces dépendances si présentes :

- spring-ai-starter-vector-store-pgvector (pgvector — non utilisé)
- spring-boot-starter-data-mongodb (MongoDB — non utilisé)

TÂCHE 2 — Conserver uniquement ces dépendances Spring AI :

- spring-ai-starter-model-google-genai (appels Gemini)
- spring-ai-tika-document-reader (extraction de texte depuis les fichiers HTML)

TÂCHE 3 — Vérifier la déclaration du BOM Spring AI :

- spring-ai-bom version 2.0.0 dans dependencyManagement

Contraintes : Ne modifier aucune autre dépendance. Retourner le pom.xml complet.

■ Sortie attendue :

Le pom.xml complet et propre. Validation : mvn dependency:tree résout spring-ai-starter-model-google-genai et spring-ai-tika sans conflit.

■ ***Conseil :*** *Vérifier également dans application.yaml que spring.ai.vectorstore n'est plus configuré (traité dans P02).*

PROMPT 02 (MISE À JOUR) — Configuration — application.yaml

■ Objectif :

Mettre à jour application.yaml et application-test.yaml : retirer le bloc pgvector et MongoDB, configurer le bloc intelligence.sec et les paramètres de summarization.

■ Contexte :

L'architecture simplifiée ne requiert aucun vector store. Un nouveau bloc intelligence.sec centralise l'URL de base SEC EDGAR et le User-Agent obligatoire.

■ Instructions & Tâches :

Tu es un expert Spring Boot. Voici les fichiers actuels : [COLLER application.yaml et application-test.yaml]

TÂCHE 1 — Modifier application.yaml :

- Supprimer spring.ai.vectorstore.pgvector et spring.data.mongodb.uri
- Conserver spring.ai.google.genai (chat)
- Ajouter à la fin :

intelligence:

```
sec:
base-url: https://data.sec.gov
www-url: https://www.sec.gov
user-agent: cairedine.finance contact@cairedine.com
summarization:
max-sections-tokens: 80000
filing-ttl-days: 90
```

TÂCHE 2 — Modifier application-test.yaml :

- Supprimer PgVectorStoreAutoConfiguration et MongoAutoConfiguration de la liste exclude.
- Conserver GoogleGenAiAutoConfiguration dans exclude.

■ Sortie attendue :

Les deux fichiers YAML complets et valides. Le contexte Spring démarre sans tentative de connexion vectorielle ou MongoDB.

■ ***Conseil :*** *Le User-Agent SEC EDGAR doit impérativement respecter le format 'OrganizationName email@domain.com' pour éviter les erreurs HTTP 403 Forbidden.*

PROMPT 03 — Structure du module intelligence (Spring Modulith)

■ Objectif :

Créer le squelette complet du module intelligence avec son package-info.java Modulith, l'arborescence hexagonale et les classes placeholder annotées.

■ Contexte :

Le projet suit l'architecture Spring Modulith. Les modules existants sont : user, shared, webclient, financialanalysis, watchlist. Le module intelligence doit déclarer ses dépendances autorisées.

■ Instructions & Tâches :

Tu es un expert Spring Modulith et architecture hexagonale. Package racine : com.cairedine.finance.app.intelligence

FICHIERS À CRÉER :

1. package-info.java (racine du module) :

@ApplicationModule(allowedDependencies = {"shared::exceptions", "shared::web", "webclient", "financialanalysis", "watchlist"})

2. Sous-packages : .domain.model, .domain.port, .application, .infrastructure.sec, .infrastructure.sec.dto, .infrastructure.persistence, .infrastructure.event, .infrastructure.web

3. Classes placeholder :

- application/ -> DocumentIngestionService (@Service @RequiredArgsConstructor @Slf4j)
- application/ -> AiSummaryServiceImpl (@Service @RequiredArgsConstructor @Slf4j)
- infrastructure.sec/ -> SecEdgarAdapter (@Component @RequiredArgsConstructor @Slf4j)
- infrastructure.sec/ -> CikResolverService (@Service @Slf4j)
- infrastructure.sec/ -> SecRestClientConfig (@Configuration)
- infrastructure.persistence/ -> InsightPersistenceAdapter (@Component @RequiredArgsConstructor)
- infrastructure.event/ -> IntelligenceEventListener (@Component)
- infrastructure.web/ -> InsightController (@RestController @RequestMapping("/api/v1/insights") @RequiredArgsConstructor)

■ Sortie attendue :

L'ensemble des fichiers .java squelettes créés. Validation : mvn compile et ApplicationModules.of(FinanceApp.class).verify() réussissent.

■ ***Conseil :*** *Ne pas implémenter les méthodes métier dans ce prompt ; l'objectif est d'établir la frontière Modulith propre.*

PROMPT 04 — Modèles de domaine + Entité JPA (Synthèse Hybride)

■ Objectif :

Implémenter les records Java du domaine et l'entité JPA PostgreSQL pour le stockage de l'analyse hybride (croisement des données quantitatives et des extraits SEC).

■ Contexte :

Deux couches distinctes : (1) Le domaine pur (records Java immuables, zéro dépendance Spring/JPA), (2) L'entité JPA qui persiste le rapport consolidé en PostgreSQL. Pour référence : FullMetrics.java, QualityMetrics.java, GrowthMetrics.java.

■ Instructions & Tâches :

PARTIE 1 — Records de domaine (package com.cairedine.finance.app.intelligence.domain.model) :

1. InsightSection.java (enum) : RISK_FACTORS, MANAGEMENT_DISCUSSION, GUIDANCE, SUPPLY_CHAIN, KEY_HIGHLIGHTS avec sectionTitle et summaryInstruction.

2. SecFiling.java (record) : String accessionNumber, String formType, String period, String rawContent, Instant fetchedAt.

3. SectionContent.java (record) : InsightSection section, String rawText, int wordCount, méthode isEmpty().

4. CompanyInsight.java (record) :

- String ticker, String formType, String period, String accessionNumber
- String synthesePerformance (Synthèse croisée globale des résultats opérationnels et financiers)
- String analyseMargesEtDette (Explication des marges réelles et du ratio Net Debt / EBITDA au vu des commentaires du management)
- List risquesPrincipaux (5 à 8 risques majeurs avec impact potentiel sur la rentabilité)
- String guidanceManagement (Perspectives et projections exprimées par les dirigeants)
- String chaineApprovisionnement (Mentions logistiques, supply chain et fournisseurs, nullable)
- List faitsMarquants (3 à 5 faits saillants de la période)
- Instant generatedAt, méthode boolean hasSupplyChainData()

PARTIE 2 — Entité JPA (package infrastructure.persistence) :

5. FinancialInsightJpaEntity.java : @Entity @Table(name="intelligence_insights", uniqueConstraints=@UniqueConstraint(columnNames={"ticker","form_type","period"}))

- Colonnes TEXT pour les champs de synthèse et JSON sérialisé pour les listes.
- Méthode isExpired() (comparaison expiresAt), @PrePersist / @PreUpdate.

■ Sortie attendue :

5 fichiers Java créés. Validation : Hibernate génère la table intelligence_insights avec les contraintes d'unicité au démarrage.

> ■ ***Conseil :*** *Utiliser un AttributeConverter, String> avec Jackson ObjectMapper pour stocker les listes JSON dans des colonnes TEXT PostgreSQL sans surcoût de jointure.*

PROMPT 05 — Interfaces de port (Architecture Hexagonale)

■ Objectif :

Définir les trois interfaces de port du domaine intelligence : source SEC EDGAR, persistance PostgreSQL, et service primaire de résumé IA.

■ Contexte :

Le domaine reste pur et découplé des détails techniques. IAiSummaryService est le port primaire utilisé par l'infrastructure web, tandis que IDocumentSourcePort et ICompanyInsightRepository sont des ports secondaires.

■ Instructions & Tâches :

Tu es un expert en architecture hexagonale. Crée les interfaces dans com.cairedine.finance.app.intelligence.domain.port :

1. IDocumentSourcePort.java :
   - List fetchFilings(String ticker, String formType, int limit) — Récupère les dépôts bruts
   - boolean isAvailable(String ticker) — Vérifie si le ticker est indexé par SEC EDGAR
2. ICompanyInsightRepository.java :
   - void save(String ticker, CompanyInsight insight) — Sauvegarde ou met à jour le résumé
   - List findByTicker(String ticker) — Retourne les insights non expirés
   - Optional findByTickerAndPeriod(String ticker, String formType, String period)
   - boolean existsByAccessionNumber(String accessionNumber) — Idempotence
3. IAiSummaryService.java (Port primaire) :
   - List getInsights(String ticker) — Lecture seule pour le contrôleur REST
   - boolean isReady(String ticker) — Indique si au moins un rapport complet est disponible

Contraintes : Interfaces Java pures, aucune annotation Spring, Javadoc soignée.

■ Sortie attendue :

3 interfaces de port compilables et cohérentes.

■ ***Conseil :*** *IAiSummaryService est le seul point d'entrée exposé pour la consultation. L'ingestion est gérée de manière interne et autonome.*

PROMPT 06 — CikResolverService + SecRestClientConfig

■ Objectif :

Implémenter le service de résolution des tickers boursiers en identifiants CIK SEC EDGAR (chargé au démarrage), et configurer les deux RestClient dédiés.

■ Contexte :

SEC EDGAR impose des CIK à 10 chiffres (ex: AAPL -> 0000320193). Le fichier officiel est https://www.sec.gov/files/company_tickers.json. Deux RestClient sont requis (data.sec.gov pour les soumissions et www.sec.gov pour les fichiers).

■ Instructions & Tâches :

FICHIER 1 — SecRestClientConfig.java (@Configuration dans infrastructure.sec) :

- @Bean RestClient secDataRestClient() avec baseUrl ${intelligence.sec.base-url}
- @Bean RestClient secWwwRestClient() avec baseUrl ${intelligence.sec.www-url}
- Header User-Agent obligatoire : ${intelligence.sec.user-agent}
- Timeouts : Connect 10s, Read 30s.

FICHIER 2 — CikResolverService.java (@Service @Slf4j dans infrastructure.sec) :

- Injecter @Qualifier("secWwwRestClient") RestClient.
- volatile Map tickerToCik = Map.of() (thread-safe pour virtual threads).
- Méthode @PostConstruct void loadMappings() : Appel GET /files/company_tickers.json, parsing Map, log du nombre d'entrées (résilient aux pannes réseau).
- String resolve(String ticker) : Renvoie le CIK ou lève TickerNotFoundException.
- boolean isKnown(String ticker) : Vérifie la présence du ticker.

■ Sortie attendue :

SecRestClientConfig.java et CikResolverService.java opérationnels avec tests mockés.

■ ***Conseil :*** *Ne jamais partager le RestClient de FMP : la SEC impose un User-Agent strictement formaté sous peine de blocage HTTP 403.*

PROMPT 07 — SecEdgarAdapter (Téléchargement 10-K et 10-Q)

■ Objectif :

Implémenter l'adaptateur SEC EDGAR qui télécharge les rapports 10-K et 10-Q et retourne leur contenu brut sous forme de SecFiling.

■ Contexte :

L'API SEC retourne les soumissions sous forme de tableaux JSON parallèles. SecEdgarAdapter implémente IDocumentSourcePort.

■ Instructions & Tâches :

Créer SecEdgarAdapter.java dans infrastructure.sec.

Dépendances : @Qualifier("secDataRestClient"), @Qualifier("secWwwRestClient"), CikResolverService.

MÉTHODE 1 — List fetchFilings(String ticker, String formType, int limit) :

- a) Résolution CIK via cikResolver.resolve(ticker)
- b) GET /submissions/CIK{cik}.json sur secDataClient -> SecSubmissionDto
- c) Filtrage des index correspondant à form == formType dans les listes parallèles
- d) Extraction accessionNumber et filingDate (limité aux `limit` plus récents)
- e) Appel de downloadFilingContent(cik, accessionNumber, formType)
- f) Retour de List (gestion des échecs individuels avec try/catch + WARN)

MÉTHODE 2 — String downloadFilingContent(String cik, String accessionNumber, String formType) (privée) :

- Parsing index JSON sur secWwwClient : /Archives/edgar/data/{cikLong}/{cleanAccession}/{accessionNumber}-index.json
- Téléchargement du fichier primaire .htm/.txt correspondant.

DTOs créés dans infrastructure.sec.dto : SecSubmissionDto, SecFilingsDto, SecRecentFilingsDto, SecFilingIndexDto, SecFilingDocDto.

■ Sortie attendue :

SecEdgarAdapter.java et les 5 DTOs de mapping SEC créés et validés par tests unitaires.

■ ***Conseil :*** *Attention au piège de l'API SEC : filings.recent.form et filings.recent.accessionNumber sont des listes synchronisées par indice (index i).*

PROMPT 08 — HtmlTextCleaner + SectionExtractor

■ Objectif :

Implémenter le nettoyeur HTML Tika et l'extracteur de sections stratégiques (Risk Factors, MD&A;, Guidance, Supply Chain) préparant le texte pour Gemini.

■ Contexte :

Les rapports SEC contiennent des dizaines de milliers de lignes HTML et de métadonnées XBRL. L'extracteur isole les sections qualitatives représentant 90% de la valeur analytique.

■ Instructions & Tâches :

CLASSE 1 — HtmlTextCleaner.java (@Component dans infrastructure.sec) :

- Méthode clean(String htmlContent) : Conversion en texte brut via TikaDocumentReader de Spring AI.
- Fallback regex si nécessaire ; suppression des balises XBRL, des tableaux de chiffres isolés et artefacts.

CLASSE 2 — SectionExtractor.java (@Component @Slf4j dans infrastructure.sec) :

- Méthode List extractSections(String cleanText) :

- RISK_FACTORS -> regex 'RISK FACTORS|ITEM 1A'

- MANAGEMENT_DISCUSSION -> regex 'MANAGEMENT\'S DISCUSSION|ITEM 7|MD&A;'

- GUIDANCE -> regex 'OUTLOOK|GUIDANCE|FORWARD-LOOKING'

- SUPPLY_CHAIN -> regex 'SUPPLY CHAIN|MANUFACTURING|SUPPLIERS'

- Découpage et extraction du contenu textuel (max 15 000 caractères par section).
- Méthode estimateTokens(String text) avec contrôle du plafond @Value("${intelligence.summarization.max-sections-tokens:80000}").

■ Sortie attendue :

HtmlTextCleaner.java et SectionExtractor.java validés par tests unitaires sur HTML de test.

■ ***Conseil :*** *Préférer un faux négatif (section absente) à un faux positif (texte incohérent). Si aucune section n'est isolée, retenir les 10 000 premiers caractères du texte nettoyé.*

PROMPT 09 — DocumentIngestionService + AiSummaryServiceImpl (Hybride Quanti+Quali)

■ Objectif :

Implémenter le pipeline complet d'ingestion et le service de synthèse IA fusionnant les métriques calculées (IFinancialAnalysisService) et les extraits SEC via Gemini.

■ Contexte :

Cœur du module. DocumentIngestionService orchestre la récupération des données quantitatives et des rapports SEC. AiSummaryServiceImpl formate le prompt croisé et appelle ChatClient Gemini pour générer le JSON structuré CompanyInsight.

■ Instructions & Tâches :

CLASSE 1 — AiSummaryServiceImpl.java (implements IAiSummaryService) :

• Constante SYSTEM_PROMPT : 'Tu es un analyste financier expert. Tu croises des métriques quantitatives réelles (marges, dette/ebitda, ROIC, croissance) avec des extraits 10-K/10-Q pour produire une synthèse financière factuelle et structurée en JSON.'

• Méthode CompanyInsight summarize(String ticker, SecFiling filing, List sections, List metrics) :

- Formate le bloc quantitatif (Marge Op, ROIC, Net Debt/EBITDA, FCF Margin, Croissance 3 ans CA/EBITDA, Multiples PE/EV).

- Formate le bloc qualitatif des sections SEC extraites.

- Injecte le schéma JSON attendu avec un exemple complet dans le user prompt.

- Appelle chatClient.prompt().system(SYSTEM_PROMPT).user(userPrompt).call().entity(CompanyInsight.class).

• getInsights(ticker) et isReady(ticker) délégués au repository.

CLASSE 2 — DocumentIngestionService.java (@Service @RequiredArgsConstructor) :

• Méthode @Async void ingestDocuments(String ticker) :

1. Vérification documentSourcePort.isAvailable(ticker)

2. Récupération des métriques : List metrics = financialAnalysisService.computeMetrics(ticker);

3. Fetch parallèle 10-K (×2) et 10-Q (×4) via CompletableFuture et virtual threads.

4. Pour chaque SecFiling : vérification idempotence, clean, extractSections, summarize croisé, save en PostgreSQL.

CLASSE 3 — InsightPersistenceAdapter.java (implements ICompanyInsightRepository) :

• Upsert dans intelligence_insights, filtrage isExpired() selon filing-ttl-days.

■ Sortie attendue :

Pipeline d'ingestion et service IA complets, testés avec Mockito sans appel réseau réel.

> ■ ***Conseil :*** *L'injection des métriques réelles dans le prompt permet au LLM d'expliquer les raisons fondamentales des variations de rentabilité et d'endettement sans halluciner sur les chiffres.*

PROMPT 10 — InsightController & DTOs Restitution (Pour Angular)

■ Objectif :

Implémenter le contrôleur REST exposant les endpoints en lecture seule servant le JSON consolidé pour l'affichage et la génération du PDF par le client Angular.

■ Contexte :

Le module n'expose que des données pré-calculées depuis PostgreSQL. Si les résumés sont en cours de génération, le contrôleur répond HTTP 202 Accepted pour permettre au client de poller.

■ Instructions & Tâches :

Créer InsightController.java dans com.cairedine.finance.app.intelligence.infrastructure.web :

ENDPOINT 1 — GET /api/v1/insights/{ticker} :

- Authentification : JwtAuthenticationToken (audit du keycloakId).
- Si insightService.isReady(ticker) == false -> Retourne HTTP 202 Accepted :

{ "status": "INGESTION_PENDING", "message": "Analyse et synthèse en cours pour {ticker}" }

- Si isReady == true -> Retourne HTTP 200 OK avec List.
- TickerNotFoundException remonte vers GlobalExceptionHandler -> HTTP 404.

ENDPOINT 2 — GET /api/v1/insights/{ticker}/status :

- Retourne HTTP 200 avec IngestionStatusResponse(ticker, ready, count, lastGenerated).

DTOs internes : InsightResponse (toutes les sections quanti/quali), IngestionStatusResponse, InsightWebMapper.

Annotations OpenAPI complètes (@Tag, @Operation, @ApiResponses, @Schema).

■ Sortie attendue :

InsightController.java et DTOs validés par test @WebMvcTest simulant les réponses 200, 202 et 404.

■ ***Conseil :*** *Le code 202 Accepted est capital pour une UX fluide dans Angular : l'UI affiche immédiatement les données de valorisation existantes tout en montrant un spinner de chargement pour la synthèse IA.*

PROMPT 11 — Intégration Événementielle Spring Modulith

■ Objectif :

Publier un événement TickerAddedEvent lors de l'ajout d'une valeur à la Watchlist et l'écouter dans le module intelligence pour déclencher l'ingestion asynchrone.

■ Contexte :

Respect strict du découplage Spring Modulith : WatchlistService ne doit pas référencer DocumentIngestionService. La communication s'effectue par événements de domaine.

■ Instructions & Tâches :

TÂCHE 1 — Créer TickerAddedEvent.java :

- Package : com.cairedine.finance.app.watchlist (contrat public du module).
- public record TickerAddedEvent(String keycloakId, String ticker) {}

TÂCHE 2 — Modifier WatchlistService.java (module watchlist) :

- Injecter ApplicationEventPublisher eventPublisher.
- Dans addTickerToWatchlist(), après la persistance réussie :

eventPublisher.publishEvent(new TickerAddedEvent(keycloakId, ticker));

TÂCHE 3 — Créer IntelligenceEventListener.java :

- Package : com.cairedine.finance.app.intelligence.infrastructure.event
- @Component, injecter DocumentIngestionService.
- @ApplicationModuleListener :

public void onTickerAdded(TickerAddedEvent event) {

```
  log.info("Ingestion déclenchée pour {} (user: {})", event.ticker(), event.keycloakId());
  try { documentIngestionService.ingestDocuments(event.ticker()); }
  catch (Exception e) { log.error("Échec ingestion pour {} : {}", event.ticker(), e.getMessage()); }
```

}

TÂCHE 4 — Vérifier package-info.java du module intelligence : autoriser la dépendance 'watchlist'.

■ Sortie attendue :

Intégration événementielle testée avec @SpringBootTest vérifiant la publication de l'événement.

■ ***Conseil :*** *@ApplicationModuleListener garantit que le traitement ne démarre qu'APRÈS le commit de la transaction d'ajout en watchlist.*

PROMPT 12 — Tests Unitaires & Intégration

■ Objectif :

Écrire les tests unitaires complets de AiSummaryServiceImpl (prompt enrichi), DocumentIngestionService (pipeline et résilience) et InsightControllerTest (@WebMvcTest).

■ Contexte :

Stack de test standardisée : JUnit 5, Mockito (@ExtendWith), AssertJ, Spring MockMvc.

■ Instructions & Tâches :

FICHIER 1 — AiSummaryServiceImplTest.java :

• Tests @Nested 'Construction du prompt' :

- shouldIncludeQuantitativeMetricsInPrompt : Vérifier via ArgumentCaptor que le prompt envoyé à Gemini contient bien les marges, le ROIC et le ratio dette/ebitda.

- shouldIncludeSecTextSectionsInPrompt : Vérifier la présence des sections MD&A; et Risk Factors.

- shouldReturnEnrichedInsightWithCorrectTicker.

• Tests @Nested 'Cas limites' : Gestion des sections vides et base sans résumés.

FICHIER 2 — DocumentIngestionServiceTest.java :

• shouldFetchMetricsAndFilingsOnIngestion : Vérifie l'appel combiné à IFinancialAnalysisService et SEC EDGAR.

• shouldSkipAlreadyIngestedFiling : Vérification de l'idempotence via existsByAccessionNumber.

• shouldContinueWhenOneFilingFails : Résilience si un filing individuel échoue.

• shouldNotFailWhenTickerNotAvailable.

FICHIER 3 — InsightControllerTest.java (@WebMvcTest) :

• shouldReturn200WithInsightsWhenReady, shouldReturn202WhenIngestionPending, shouldReturn404ForUnknownTicker, shouldReturn401WithoutJwt.

■ Sortie attendue :

3 fichiers de test avec 100% de réussite : mvn test -Dtest="AiSummaryServiceImplTest,DocumentIngestionServiceTest,InsightControllerTest".

■ ***Conseil :*** *Utiliser ChatClientBuilder dans les tests Mockito pour mocker élégamment l'interface fluide de Spring AI.*

4. Annexes Techniques

Annexe A — Variables d'environnement (.env)

```
# Google AI Studio (Gemini 2.0 Flash) GEMINI_API_KEY=AIzaSy...votre-cle-api # PostgreSQL (Identique à la configuration existante) SPRING_DATASOURCE_URL=jdbc:postgresql://localhost:5432/finance_db SPRING_DATASOURCE_USERNAME=postgres SPRING_DATASOURCE_PASSWORD=postgres
```

Annexe B — Schéma de la table PostgreSQL

```
CREATE TABLE intelligence_insights ( id BIGSERIAL PRIMARY KEY, ticker VARCHAR(20) NOT NULL, form_type VARCHAR(10) NOT NULL, period VARCHAR(20) NOT NULL, accession_number VARCHAR(30), synthese_performance TEXT, analyse_marges_et_dette TEXT, risques_principaux TEXT, -- JSON Array sérialisé guidance_management TEXT, chaine_approvisionnement TEXT, faits_marquants TEXT, -- JSON Array sérialisé generated_at TIMESTAMPTZ NOT NULL, expires_at TIMESTAMPTZ NOT NULL, UNIQUE (ticker, form_type, period) );
```

Annexe C — Protocole de validation de bout en bout

1. **Compilation** : mvn clean compile (zéro avertissement ou erreur de frontière Modulith).
2. **Tests automatisés** : mvn test (100% au vert sur l'ensemble de l'application).
3. **Vérification Modulith** : ApplicationModules.of(FinanceApp.class).verify() validé sans cycles.
4. **Déclenchement** : POST /api/v1/watchlist/AAPL ajoute le ticker et émet TickerAddedEvent.
5. **Vérification statut initial** : GET /api/v1/insights/AAPL/status retourne { ready: false, count: 0 } (HTTP 200) ou GET /api/v1/insights/AAPL retourne HTTP 202.
6. **Traitement en arrière-plan** : Téléchargement SEC EDGAR (10-K x2, 10-Q x4), calcul des métriques et appels Gemini (15 à 30 secondes).
7. **Vérification statut final** : GET /api/v1/insights/AAPL/status retourne { ready: true, count: 6 }.
8. **Restitution & Rapport** : GET /api/v1/insights/AAPL fournit le JSON consolidé complet. Le frontend Angular l'affiche et génère le rapport PDF instantanément au clic de l'utilisateur.